

Hippocampal Neuropixels Simulation and Causal Manifold Decoding

Technical project document

hippo repository

July 20, 2026

Abstract

This document specifies a Python research pipeline that (i) simulates hippocampal and afferent single-unit activity during open-field navigation, (ii) records it through a Neuropixels-like probe with sorting degradation, and (iii) decodes behavioral latent variables from *causal* population spike counts. The recommended neural backend uses a maximally hippocampal RatInABox population table (CA1–CA3, DG, subiculum, MEC) with anatomically correct probe contact mapping and post-hoc theta / ripple / sparsity / recurrent / inhibitory overlays. Hyperparameter search over decoder models, integration windows, and optional manifold features is performed automatically; the selected configuration is then replayed in a closed-loop causal setting. We document rate equations, probe–region geometry, implemented and planned inter-region coupling, causal decoding, decoder models, and manifold feature modes with explicit mathematics and default parameters.

Contents

1	Introduction and scientific framing	3
2	Public software workflow	3
3	Simulation pipeline	4
3.1	Neural backends	4
4	Neural rate model	4
4.1	Dynamics	4
4.2	Target drive	5
4.3	Default parameters	5
5	Maximally hippocampal RatInABox populations	6
6	Probe anatomy and region contact	7
6.1	Cell type $\leftrightarrow$ probe contact	7
7	Hippocampal dynamics overlays	8
7.1	Shared global features	8
7.2	Per-group overlays	8
7.3	CA1 interneuron equation	8
8	Inter-region coupling: implemented and planned	8
8.1	What is implemented today	8
8.2	Derived target: trisynaptic feedforward	9
8.3	Decoder-facing consequence	9

9	Causal decoding	9
9.1	Observation model	9
9.2	Timing symbols	9
9.3	Architectural separation	10
9.4	Decoding targets	10
10	Decoder model zoo	10
10.1	Quick mode (default)	10
10.2	Full mode	10
10.3	Selection policies	10
11	Bayesian place decoder	11
12	Manifold features	11
12.1	Static manifold-informed decoding	11
12.2	Optional temporal manifold decoding	11
13	Representative results from the latest run	11
13.1	Simulation and recording summary	12
13.2	Manifold-informed decoding	14
13.3	Offline decoder selection and recording degradation	16
13.4	Causal realtime replay	17
14	Outputs and visualization	18
15	Future RatInABox behavior extensions	18
15.1	Alignment with RatInABox	19
15.2	Real trajectory implementation (<code>import_trajectory</code>)	19
15.3	RL-based reward simulation	19
16	Limitations	19
17	Repository map (selected)	20

1 Introduction and scientific framing

The simulator knows the true behavioral state. The decoder observes only spike times (ground-truth Poisson spikes or spikes after Neuropixels-like degradation and sorting). Comparing these two spike sources quantifies information loss due to extracellular recording and sorting. Searching causal integration windows asks how much recent neural history is required to decode spatial, proprioceptive, and contextual variables. Searching decoder families asks which estimator best extracts those latents from population activity. Closed-loop replay tests whether decoded estimates can drive realtime triggers.

Six computational operations are kept conceptually separate:

1. population partition (which units enter a model);
2. observation construction (causal spike counts);
3. manifold geometry (map to latents z_t);
4. temporal dynamics on the manifold (history of length L);
5. behavioral decoding ($\hat{y}_{t+\tau}$);
6. inter-region coupling (partially implemented; trisynaptic feedforward planned; Section 8).

2 Public software workflow

The intended user interface consists of three scripts:

1. `run_simulation.py` — generate behavior, rates, spikes, recording, sorting;
2. `run_decoder.py` — compare models/windows, select the best configuration, run closed-loop replay, optionally plot (calls `run_full_decoder_workflow`);
3. `run_visualizations.py` — regenerate figures from saved outputs only (never retrains).

Listing 1: Standard end-to-end commands.

```
python run_simulation.py \  
  --output outputs/ratinabox_004 \  
  --seed 4 \  
  --neural-backend ratinabox_neurons  
  
python run_decoder.py \  
  --input outputs/ratinabox_004 \  
  --output outputs/ratinabox_004 \  
  --compare-sources \  
  --closed-loop-target spatial_context \  
  --selection-policy shortest_near_optimal \  
  --enable-temporal-manifold \  
  --compile-pdf  
  
python run_visualizations.py \  
  --experiment outputs/ratinabox_004 \  
  --all --compile-pdf
```

Hyperparameter tuning is intentionally *internal* to the decoder workflow: users need not manually choose W or call separate comparison/replay scripts for the standard path.

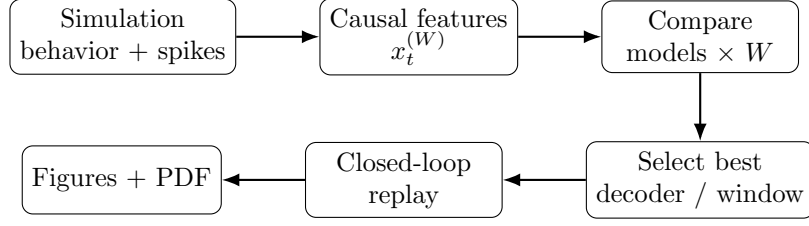


Figure 1: High-level pipeline. Tuning and selection occur under the hood before realtime replay.

3 Simulation pipeline

Default session duration is 600 s (10 minutes). Behavior and rate updates run at 20 Hz ($\Delta t = 0.050$ s).

1. **Behavior:** RatInABox open-field trajectory (thigmotaxis, stalls, smooth turns).
2. **Features:** place, head direction, speed, acceleration, boundary distance, theta phase, ripple.
3. **Rates:** custom CA1/CA2/CA3/DG equations *or* RatInABox neuron classes.
4. **Spikes:** inhomogeneous Poisson sampling (ground truth).
5. **Recording:** multi-channel templates, noise, motion amplitude drift, collisions.
6. **Sorting:** Kilosort-like re-extraction with misses, jitter, contamination.

3.1 Neural backends

`custom_rate_equations` Explicit hippocampal rate model (Section 4), now including MEC / subiculum / CA1 interneuron parameter blocks.

`ratinbox_neurons` **Recommended.** Maximally hippocampal RatInABox population table with anatomical probe contact mapping and hippocampal dynamics overlays (Sections 5–8).

Primary simulation outputs: `behavior.csv`, `units.csv`, `spikes_ground_truth.csv`, `spikes_sorted.csv`, `anatomy_regions.csv`, `summary.json`, plus (RiaB) `ratinbox_group_metadata.csv`, `rates.npy`, and `neural_backend_metadata.json`.

4 Neural rate model

Source of truth in code: `hippo_sim/rate_equations.py`, `hippo_sim/config.py`. The custom backend now also carries parameter blocks for `CA1_int`, `MEC_grid`, `MEC_hd`, `MEC_speed`, and `Sub_bvc` (Table 2); the recommended path remains the RiaB backend with dynamics overlays.

4.1 Dynamics

For each unit i , rates $R_i(t)$ obey the linear filter

$$\tau_i \frac{dR_i}{dt} = -R_i + \lambda_i^{\text{target}}(t), \quad (1)$$

integrated with Euler steps at the behavioral timestep $\Delta t = 0.05$ s.

4.2 Target drive

Let $p(t)$ be allocentric position, $\theta(t)$ head direction, $v(t)$ speed, $a(t)$ acceleration, $d_{\text{wall}}(t)$ distance to the nearest wall, $\phi_\theta(t)$ theta phase, and $r(t)$ a ripple envelope. Define

$$f_i^{\text{place}}(t) = \exp\left(-\frac{\|p(t) - p_{0,i}\|^2}{2\sigma_i^2}\right), \quad (2)$$

$$f_i^{\text{HD}}(t) = \frac{\exp(\kappa_i \cos(\theta(t) - \theta_i^{\text{pref}}))}{\exp(\kappa_i)/I_0(\kappa_i)}, \quad (3)$$

$$f_i^{\text{speed}}(t) = \frac{\max(0, v(t) - v_i^{\text{thresh}})}{30 - v_i^{\text{thresh}}}, \quad (4)$$

$$f_i^{\text{acc}}(t) = \text{clip}(|a(t)|/50, 0, 1), \quad (5)$$

$$f_i^{\text{bnd}}(t) = \exp\left(-\frac{d_{\text{wall}}(t)^2}{2 \cdot 15^2}\right), \quad (6)$$

$$f_i^\theta(t) = 1 + w_i^\theta \cos \phi_\theta(t). \quad (7)$$

The multiplicative drive is

$$\begin{aligned} \text{drive}_i(t) = & b_i + A_i f_i^{\text{place}} (1 + w_i^{\text{HD}} f_i^{\text{HD}}) (1 + w_i^{\text{speed}} f_i^{\text{speed}}) (1 + 0.2 f_i^{\text{acc}}) f_i^\theta \\ & + w_i^{\text{bnd}} A_i f_i^{\text{bnd}} + w_i^{\text{ripple}} A_i r(t). \end{aligned} \quad (8)$$

Region-specific modifications:

- **CA3:** $\text{drive}_i \leftarrow \text{drive}_i + w_i^{\text{rec}} \bar{R}(t)$.
- **DG sparsity:** if $f_i^{\text{place}} (1 + w_i^{\text{speed}} f_i^{\text{speed}}) < \theta_i^{\text{sparse}}$, then $\text{drive}_i \leftarrow 0.1 b_i$.

Finally

$$\lambda_i^{\text{target}}(t) = \max(\text{drive}_i(t) \xi(t) g_i(t), 0), \quad (9)$$

where $\xi(t)$ is a shared arousal/state Ornstein–Uhlenbeck (OU) factor and $g_i(t)$ is a per-unit gain OU process.

4.3 Default parameters

Table 1: Rate parameters by principal cell type (hippocampus proper).

Parameter	CA1_pyr	CA2_pyr	CA3_pyr	DG_granule	Units
τ	0.05	0.05	0.06	0.08	s
b (baseline)	0.5	0.4	0.3	0.05	Hz
A (amplitude)	12.0	10.0	8.0	15.0	Hz
σ_{place}	10	8	12	6	cm
w_{HD}	0.4	0.35	0.2	0.1	—
κ_{HD}	2.0	2.5	1.5	1.0	—
w_{speed}	0.3	0.25	0.2	0.5	—
v_{thresh}	2	2	2	3	cm/s
w_θ	0.25	0.15	0.1	0.05	—
w_{ripple}	0.5	0.1	0.6	0.0	—
w_{boundary}	0.2	0.15	0.1	0.3	—
$w_{\text{recurrent}}$	—	—	0.15	—	—
θ_{sparse}	—	—	—	0.3	—

Table 2: Rate parameters for afferent / inhibitory types (custom backend and RiaB overlays).

Parameter	CA1_int	MEC_grid	MEC_hd	Sub_bvc	Units
τ	0.03	0.05	0.04	0.05	s
b (baseline)	18.0	0.3	0.5	0.4	Hz
A (amplitude)	4.0	12.0	10.0	10.0	Hz
σ_{place}	25	15	40	20	cm
w_{HD}	0.05	0.1	1.0	0.15	—
κ_{HD}	1.0	1.0	3.0	1.5	—
w_{speed}	0.1	0.4	0.1	0.15	—
w_{θ}	0.4	0.2	0.2	0.1	—
w_{ripple}	0.3	0.0	0.0	0.0	—
w_{boundary}	0.05	0.1	0.05	0.8	—

Table 3: Drift, recording, and sorting defaults.

Parameter	Value	Units / notes
Place drift SD	0.1	cm/min
Place update interval	30	s
State OU τ, σ	120, 0.15	s, —
Gain OU τ, σ	180, 0.1	s, —
Channels / pitch	384 / 20	— / μm
Sample rate	30	kHz
Template span	3–10	channels
Amplitude range	20–200	μV
Noise SD	15	μV
Miss rate / jitter	0.12 / 0.3	— / ms
Contamination / merge	0.08 / 0.04	—

5 Maximally hippocampal RatInABox populations

Source of truth: `hippo_sim/hippocampal_populations.py`, executed by `hippo_sim/ratinabox_neural_backend.py`.

Earlier RiaB mappings placed grid / HD / BVC / speed cells into CA1/CA2/CA3/DG as fillers. The current table remaps each RatInABox class onto a biologically plausible region and cell type, then overlays hippocampal dynamics that RiaB does not provide natively.

Table 4: Default maximally hippocampal population table (305 units).

Group	n	RiaB class	Cell type	Dynamics
CA1_place_pp	60	PhasePreprocessingPlaceCells	CA1_pyr	phase prec. (built-in), ripple, speed
CA3_place	40	PlaceCells (wide)	CA3_pyr	θ , ripple, recurrent
DG_place	50	PlaceCells (narrow)	DG_granule	θ , sparsity, speed
CA2_place	20	PlaceCells	CA2_pyr	θ , weak ripple
MEC_grid	40	GridCells	MEC_grid	θ , speed
MEC_hd	30	HeadDirectionCells	MEC_hd	θ
Sub_bvc	30	BoundaryVectorCells	Sub_bvc	θ
MEC_speed	20	SpeedCells_fallback	MEC_speed	speed tuning
CA1_int	15	synthetic	CA1_int	θ , ripple, anti-CA1

Mapping rationale:

- Place / phase-preprocessing place $\rightarrow$ CA1 / CA2 / CA3 / DG (hippocampus proper).
- Grid / HD / speed $\rightarrow$ MEC (entorhinal afferents), not CA fillers.

- Boundary vector $\rightarrow$ subiculum (boundary-rich), not CA3.
- CA1 oriens interneurons $\rightarrow$ synthetic inhibitory population.

`units.csv` stores `rate_model` and `ratinabox_class` per unit. Metadata includes `ratinabox_population_t` and `apply_hippocampal_dynamics`.

6 Probe anatomy and region contact

Neuropixels 1.0 single-shank geometry along a dorsal hippocampal depth axis, extended ventrally with co-recorded afferent bands (subiculum, MEC). Channel pitch is $20\ \mu\text{m}$ (384 sites). Exact per-run mapping is written to `anatomy_regions.csv`.

Table 5: Region segments and default cell-type contact (depth axis).

Region	Layer	Depth (μm)	Dens.	Cell type
CA1	oriens	0–200	2.0	CA1_int
CA1	pyramidal	200–400	8.0	CA1_pyr
CA1	radiatum	400–600	5.0	CA1_pyr
CA2	pyramidal	600–800	6.0	CA2_pyr
CA3	pyramidal	800–1300	7.0	CA3_pyr
DG	granule	1300–1650	10.0	DG_granule
DG	hilus	1650–1850	3.0	DG_granule
Subiculum	pyramidal	1850–2050	4.0	Sub_bvc
MEC	layer2	2050–2350	5.0	MEC_grid
MEC	layer3	2350–2600	4.0	MEC_hd

6.1 Cell type $\leftrightarrow$ probe contact

For the RiaB backend, each unit is assigned a probe channel from the depth band matching its cell type via `CELL_TYPE_TO_PROBE` in `hippo_sim/anatomy.py`:

$$(\text{region}, \text{layer}) = \begin{cases} (\text{CA1}, \text{oriens}) & \text{CA1_int}, \\ (\text{CA1}, \text{pyramidal}) & \text{CA1_pyr}, \\ (\text{CA2}, \text{pyramidal}) & \text{CA2_pyr}, \\ (\text{CA3}, \text{pyramidal}) & \text{CA3_pyr}, \\ (\text{DG}, \text{granule}) & \text{DG_granule}, \\ (\text{Subiculum}, \text{pyramidal}) & \text{Sub_bvc}, \\ (\text{MEC}, \text{layer2}) & \text{MEC_grid}, \text{MEC_speed}, \\ (\text{MEC}, \text{layer3}) & \text{MEC_hd}. \end{cases} \quad (10)$$

Channel c is drawn uniformly from sites with depth $z \in [z_{\text{start}}, z_{\text{end}})$; depth is $z = c \cdot 20\ \mu\text{m}$. Layer matching is preferred; if the exact layer pool is empty, any segment of the same region is used as fallback.

Caveat: MEC/subiculum bands are co-recorded afferent populations for decoding and manifold analysis. This is *not* a claim that a single Neuropixels shank spans all of these structures in vivo; it is a contact-assignment convenience so afferent cell types land on distinct depth bands rather than polluting CA depths.

7 Hippocampal dynamics overlays

Let $R_i^{\text{RiaB}}(t)$ be the raw RatInABox (or fallback) rate for unit i of cell type c in population group G . When `apply_hippocampal_dynamics=True` (default), the backend applies the overlays below (implemented in `apply_hippocampal_dynamics`).

7.1 Shared global features

From the same behavioral trajectory, global features include theta phase $\phi_\theta(t)$, ripple envelope $r(t) \in [0, 1]$, and speed $v(t)$. These are shared across regions (common clock / state), then weighted per cell type.

7.2 Per-group overlays

With cell-type parameters from Table 1 / 2,

$$R_i \leftarrow R_i^{\text{RiaB}}, \quad (11)$$

$$R_i \leftarrow R_i (1 + w_c^\theta \cos \phi_\theta(t)) \quad \text{if group enables theta,} \quad (12)$$

$$R_i \leftarrow R_i (1 + w_c^{\text{speed}} f^{\text{speed}}(t)) \quad \text{if speed gain,} \quad (13)$$

$$R_i \leftarrow R_i + w_c^{\text{ripple}} A_c r(t) \quad \text{if ripple,} \quad (14)$$

$$R_i \leftarrow \begin{cases} 0.1 b_c & \text{if } R_i / \max_t R_i < \theta_c^{\text{sparse}}, \\ R_i & \text{otherwise} \end{cases} \quad \text{if DG sparsity,} \quad (15)$$

$$R_i \leftarrow R_i + w_c^{\text{rec}} \bar{R}_G(t) \quad \text{if CA3 recurrent,} \quad (16)$$

where $\bar{R}_G(t) = \frac{1}{|G|} \sum_{j \in G} R_j(t)$ is the within-group population mean (CA3 autoassociative sketch). CA1 phase-precessing place cells skip the multiplicative theta overlay because phase precession is already built into the RiaB contrib class. Rates are finally clipped: $R_i \leftarrow \max(R_i, 0)$.

7.3 CA1 interneuron equation

Synthetic oriens interneurons are not a RiaB class. With baseline b_{int} , anti-CA1 weight w_{anti} (default 0.35), and normalized CA1 pyramidal population mean $\tilde{R}_{\text{CA1}}(t) = \bar{R}_{\text{CA1}}(t) / \max_t \bar{R}_{\text{CA1}}$,

$$R_i^{\text{int}}(t) = g_i b_{\text{int}} (1 + w_{\text{int}}^\theta \cos(2\pi f_\theta t)) (1 - w_{\text{anti}} \tilde{R}_{\text{CA1}}(t)) + w_{\text{int}}^{\text{ripple}} A_{\text{int}} r(t), \quad (17)$$

with per-unit gain $g_i \sim \text{Unif}(0.8, 1.2)$. This is the first explicit *cross-population* coupling in the RiaB backend (inhibitory anti-correlation with CA1 pyramids).

8 Inter-region coupling: implemented and planned

8.1 What is implemented today

Coupling currently takes three forms:

1. **Shared state:** all regions see the same $\phi_\theta(t)$, $r(t)$, and $v(t)$ (Eqs. 12–14).
2. **Within-region recurrence:** CA3 population-mean boost (Eq. 16).
3. **Cross-population inhibition:** $\text{CA1.int} \leftarrow f(\bar{R}_{\text{CA1}})$ (Eq. 17).

Afferent populations (MEC grid/HD/speed, subiculum BVC) are co-recorded on the probe but do *not* yet drive hippocampal rates through synaptic weights. Their influence on decoding is observational (they appear in $x_t^{(W)}$ and in region-/cell-type PCA partitions).

8.2 Derived target: trisynaptic feedforward

A minimal next step is to let region means modulate downstream targets. Define region (or group) means

$$\bar{R}_A(t) = \frac{1}{|A|} \sum_{i \in A} R_i(t), \quad A \in \{\text{MEC}, \text{DG}, \text{CA3}, \text{CA1}, \text{Sub}\}. \quad (18)$$

A linear feedforward sketch of the classic trisynaptic loop plus direct EC→CA1 is

$$\lambda_i^{\text{DG}} \leftarrow \lambda_i^{\text{DG}, \text{RiaB}} + w_{\text{MEC} \rightarrow \text{DG}} \bar{R}_{\text{MEC}}(t), \quad (19)$$

$$\lambda_i^{\text{CA3}} \leftarrow \lambda_i^{\text{CA3}, \text{RiaB}} + w_{\text{DG} \rightarrow \text{CA3}} \bar{R}_{\text{DG}}(t) + w_{\text{CA3} \rightarrow \text{CA3}} \bar{R}_{\text{CA3}}(t), \quad (20)$$

$$\lambda_i^{\text{CA1}} \leftarrow \lambda_i^{\text{CA1}, \text{RiaB}} + w_{\text{CA3} \rightarrow \text{CA1}} \bar{R}_{\text{CA3}}(t) + w_{\text{MEC} \rightarrow \text{CA1}} \bar{R}_{\text{MEC}}(t) - w_{\text{int} \rightarrow \text{CA1}} \bar{R}_{\text{int}}(t). \quad (21)$$

Suggested defaults (tunable hypotheses, not fitted biology): $w_{\text{MEC} \rightarrow \text{DG}} = 0.2$, $w_{\text{DG} \rightarrow \text{CA3}} = 0.25$, $w_{\text{CA3} \rightarrow \text{CA3}} = 0.15$ (already present), $w_{\text{CA3} \rightarrow \text{CA1}} = 0.2$, $w_{\text{MEC} \rightarrow \text{CA1}} = 0.15$, $w_{\text{int} \rightarrow \text{CA1}} = 0.3$. Implementation path: extend `apply_hippocampal_dynamics` (or a post-pass over stacked group rates) after all groups are computed, then re-apply nonnegativity. `RatInABox FeedForwardLayer` is an alternative if synaptic mixing should live inside `RiaB` rather than as a Hippo overlay.

8.3 Decoder-facing consequence

Until Eqs. 19–21 are implemented, region-PCA and cell-type partitions still recover *co-recorded* structure, but not true directed communication. Closed-loop decoding therefore measures information available in the joint shank population, not causal trisynaptic transfer.

9 Causal decoding

9.1 Observation model

At each behavioral frame time t , the decoder builds a causal population count vector

$$x_t^{(W)} \in \mathbb{R}^N, \quad (x_t^{(W)})_i = \#\{\text{spikes of unit } i \text{ in } [t - W, t)\}. \quad (22)$$

The interval is half-open: spikes with time $\geq t$ are never included. Optional rate features are $r_t^{(W)} = x_t^{(W)} / W$.

9.2 Timing symbols

Table 6: Timing notation.

Symbol	Default	Meaning
t / behavior timestamps	from <code>behavior.csv</code>	Prediction grid
dt_{update}	0.050 s (20 Hz)	One prediction per frame
W	searched	Causal integration window
z_t	—	Current manifold coordinate
L	optional	Latent history length (frames)
τ	0 (searchable)	Prediction lag for $\hat{y}_{t+\tau}$
train fraction	0.70	Contiguous temporal split

Default window grid:

$$W \in \{0.050, 0.100, 0.250, 0.500, 1.000\} \text{ s}. \quad (23)$$

Short W lowers latency but yields fewer spikes; long W improves count reliability at the cost of effective delay.

9.3 Architectural separation

$$x_t^{(W)} \longrightarrow z_t \longrightarrow Z_t^{(L)} = [z_{t-L+1}, \dots, z_t] \longrightarrow \hat{y}_{t+\tau}. \quad (24)$$

Here W provides spike evidence for the *current* state; L models recent movement through the manifold; τ is a future prediction horizon. These roles must not be collapsed into a single “decoder memory” parameter.

9.4 Decoding targets

Table 7: Targets and primary selection metrics.

Target	Family	Primary metric	Direction
position	continuous (2D)	mean position error (cm)	lower
speed	continuous	R^2	higher
acceleration	continuous	R^2	higher
head_direction	circular	mean circular error (deg)	lower
distance_to_wall	continuous	R^2	higher
spatial_context	categorical	balanced accuracy	higher
movement_state	categorical	balanced accuracy	higher
wall_distance_bin	categorical	balanced accuracy	higher

10 Decoder model zoo

Implemented in `realtime/decoder_models.py`.

10.1 Quick mode (default)

- Regression: Ridge ($\alpha = 1$), PCA-Ridge (95% variance), Random Forest (100 trees, depth 12).
- Classification: Logistic Regression (balanced), Random Forest classifier (balanced).
- Bayesian place decoder when applicable (Section 11).

10.2 Full mode

Additionally includes Elastic Net ($\alpha = 0.1$, ℓ_1 -ratio 0.5), PLS (10 components), histogram gradient boosting, k -NN ($k = 15$), linear SVM, and a causally smoothed Bayesian place decoder.

10.3 Selection policies

After evaluating all (model, W , feature_mode) combinations per target:

`best_accuracy` pick the best primary metric;

`shortest_near_optimal` (default) pick the shortest W within 5% of the best score ($\leq 1.05 \times$ best if lower-is-better; $\geq 0.95 \times$ best if higher-is-better).

Closed-loop replay loads the selected serialized model and does not retrain when comparison artifacts already exist.

11 Bayesian place decoder

A Poisson likelihood place-map decoder on a $n_{\text{bins}} \times n_{\text{bins}}$ grid ($n_{\text{bins}} = 20$ by default). Occupancy is floored at 10^{-3} . Optional causal posterior smoothing uses weight $\alpha = 0.7$ on the previous posterior only (never future). Schematically,

$$\log P(\text{bin} \mid x_t) \propto \log P(\text{bin}) + \sum_i \left[x_{t,i} \log \lambda_i(\text{bin}) - \lambda_i(\text{bin}) W \right]. \quad (25)$$

A derived-context variant maps the place posterior to categorical spatial / wall-distance labels.

12 Manifold features

12.1 Static manifold-informed decoding

$$x_t^{(W)} \xrightarrow{E} z_t \xrightarrow{\text{decoder}} \hat{y}_t. \quad (26)$$

The encoder E is fit on the training split only and frozen for test / realtime.

Table 8: Static feature modes (`realtime/manifold_features.py`).

Mode	Transform	Grouping	Latent dim
<code>counts</code>	$z = x$	—	N
<code>rates</code>	$z = x/W$	—	N
<code>global_pca</code>	PCA	all units	k (default 3)
<code>region_pca</code>	PCA per region	<code>region</code>	k / group
<code>layer_pca</code>	PCA per layer	<code>layer</code>	k / group
<code>cell_type_pca</code>	PCA per type	<code>cell_type</code>	k / group
<code>rate_model_pca</code>	PCA per model	<code>rate_model</code>	k / group

Quick default modes: `counts`, `global_pca`, `region_pca`.

12.2 Optional temporal manifold decoding

Enabled with `--enable-temporal-manifold`:

$$x_t^{(W)} \rightarrow z_t \rightarrow Z_t^{(L)} = [z_{t-L+1}, \dots, z_t] \rightarrow \hat{y}_{t+\tau}. \quad (27)$$

Defaults: representations `{raw,pca}` with $\sqrt{\text{counts}}$ + standardization on the temporal PCA path; history lengths $L \in \{1, 2, 5, 10, 20\}$ frames (50 ms–1 s at 20 Hz); models `raw_static`, `static_latent`, `flattened_history`; controls: averaged and time-shuffled histories. Selection uses contiguous train/validation/test splits with a leakage gap; validation only is used for model selection. Planned extensions: causal GRU/TCN, autoencoders, CEBRA.

13 Representative results from the latest run

The figures below were generated from an earlier experiment run (`ratinbox_003`). They remain representative of the decoding workflow; the simulation configuration has since moved to the maximally hippocampal RiaB table (default run `ratinbox_004`, 305 units, seed 4). Regenerate figures with `run_visualizations.py --experiment outputs/ratinabox_004 --all` after decoder artifacts exist.

13.1 Simulation and recording summary

Hippocampal Neuropixels simulation — report summary

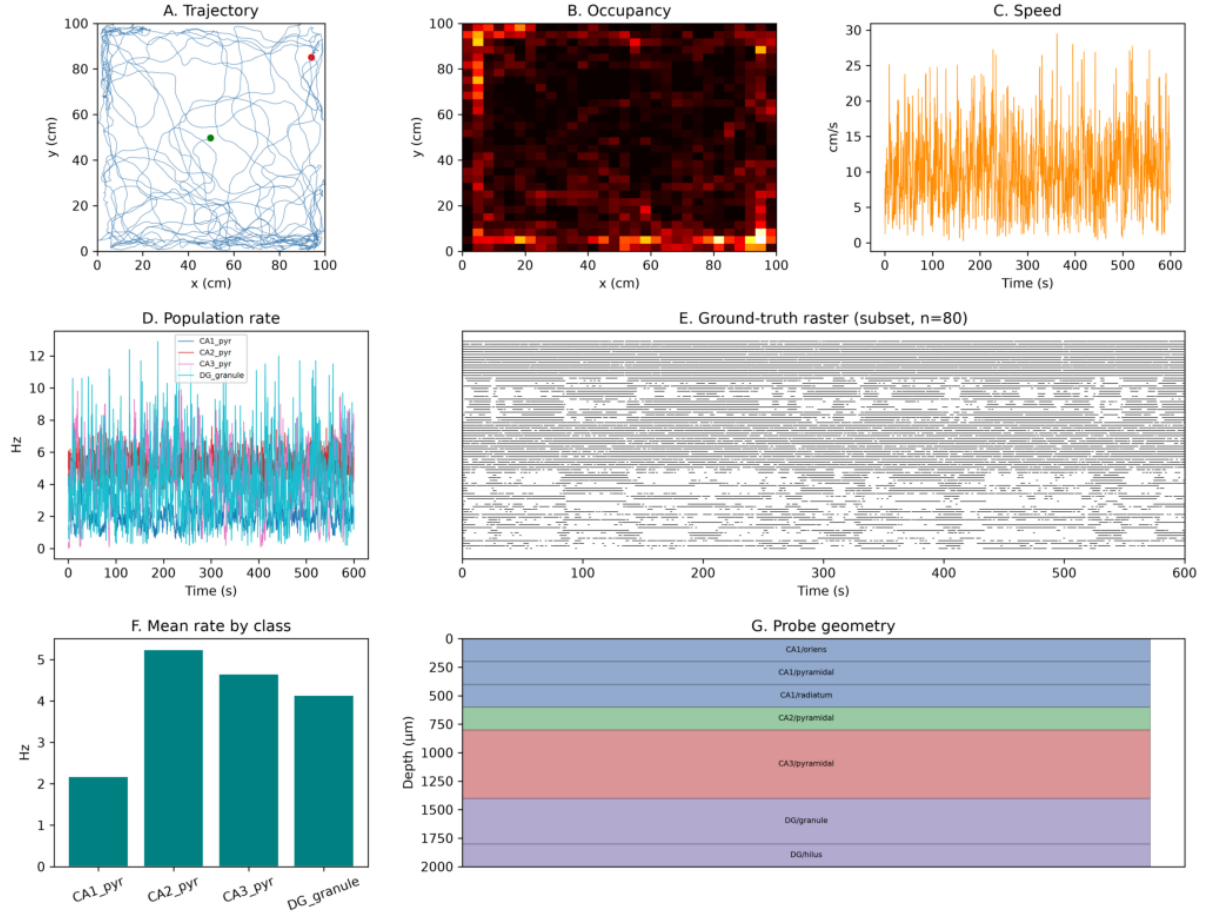


Figure 2: Composite summary of the latest hippocampal Neuropixels simulation, including trajectory, occupancy, speed, population rates, a ground-truth raster subset, mean firing rate by cell class, and probe geometry.

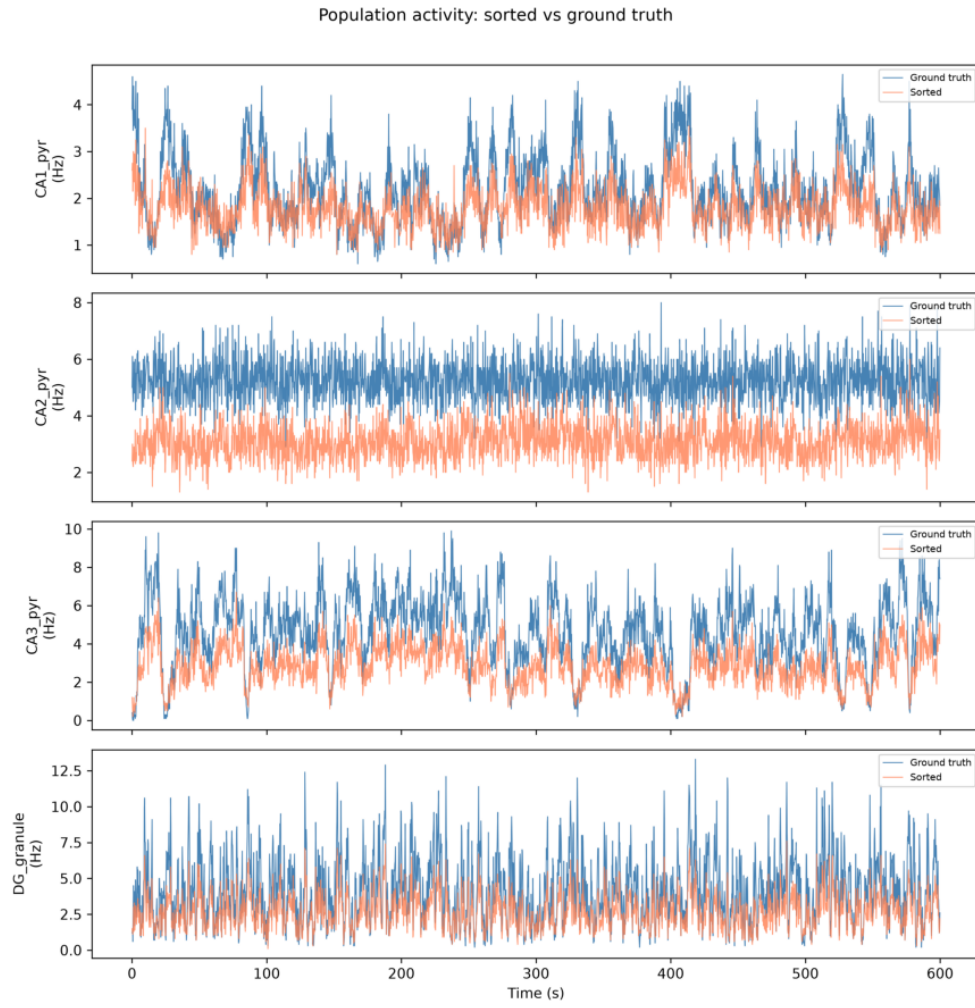


Figure 3: Population activity reconstructed from sorted spikes compared with the ground-truth population activity for each hippocampal cell class.

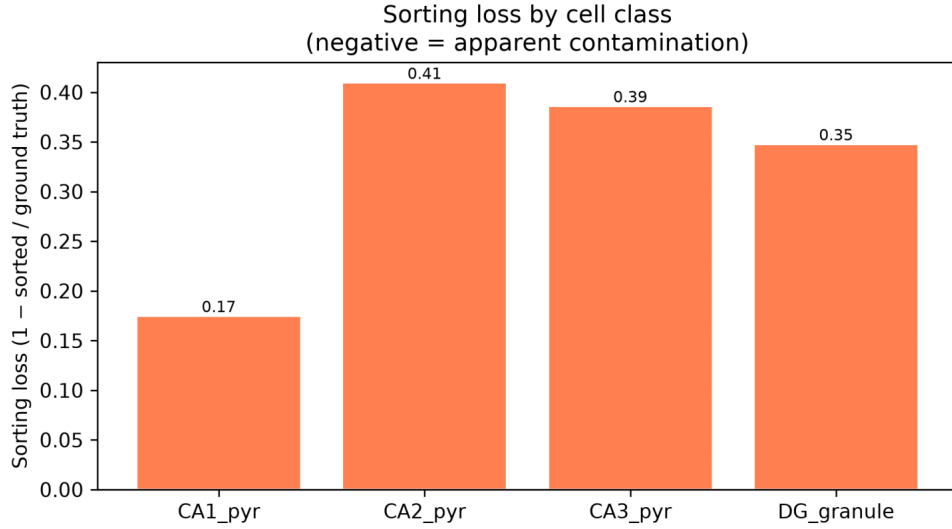


Figure 4: Relative spike-count loss after Neuropixels-like recording degradation and Kilosort-like re-extraction. The latest run shows the smallest loss for CA1 pyramidal units and larger losses for CA2, CA3, and DG populations.

13.2 Manifold-informed decoding

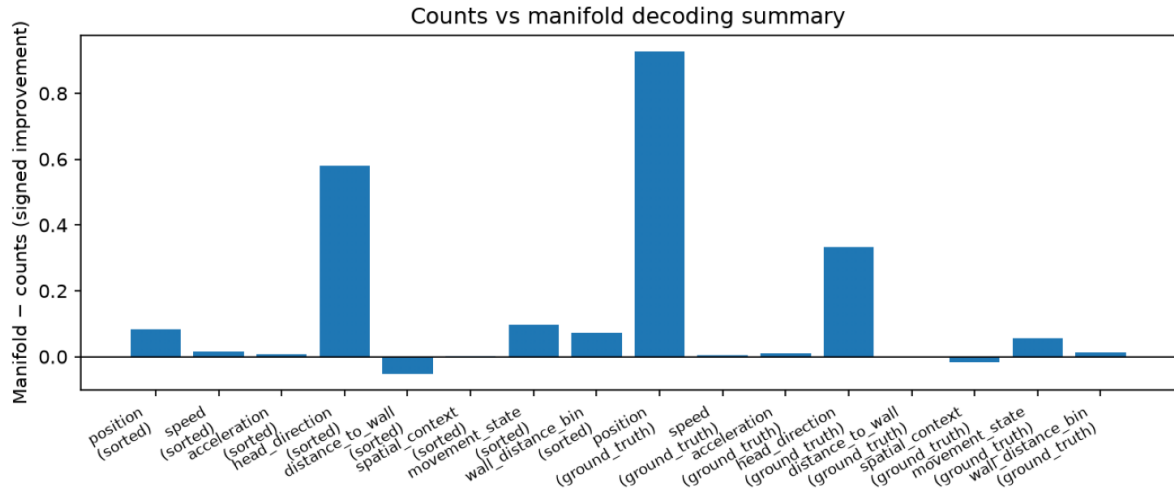


Figure 5: Signed performance change for manifold features relative to raw spike counts across behavioral targets and spike sources. Positive values indicate improvement from manifold representation.

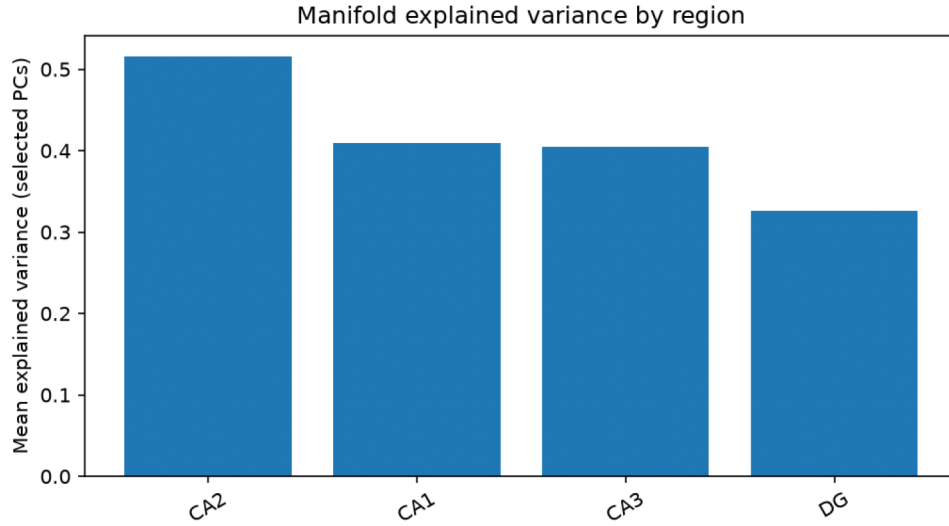


Figure 6: Mean variance explained by the selected region-level PCA components. CA2 has the most compact low-dimensional representation in this run, while DG requires a larger fraction of dimensions to capture comparable variance.

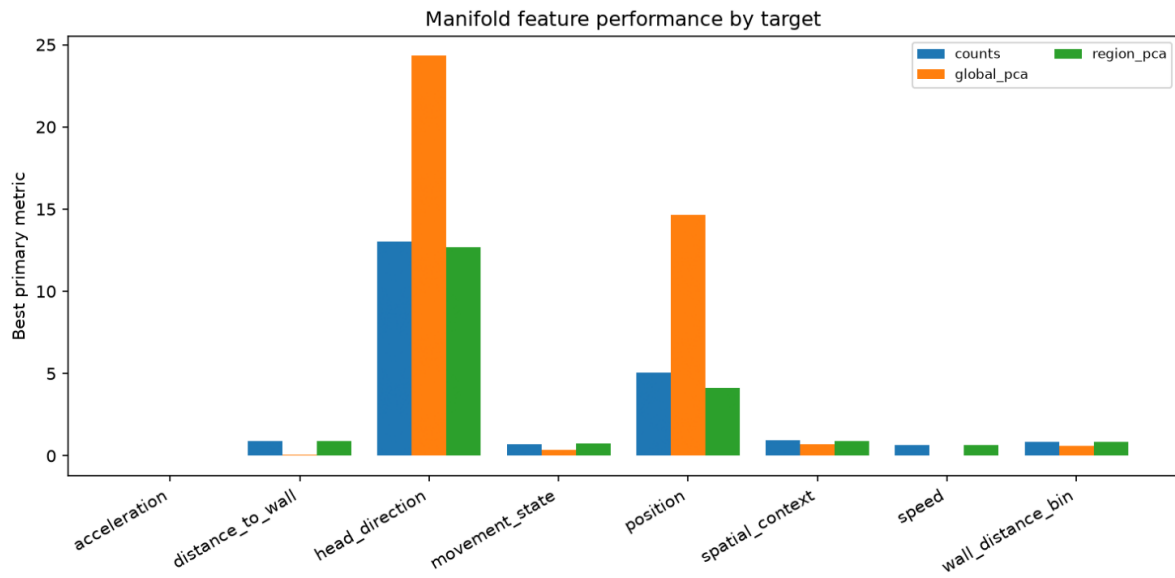


Figure 7: Best decoding metric by target for raw counts, global PCA, and region-wise PCA features. Metric direction differs by target, so this panel should be interpreted together with the target definitions in Table 7.

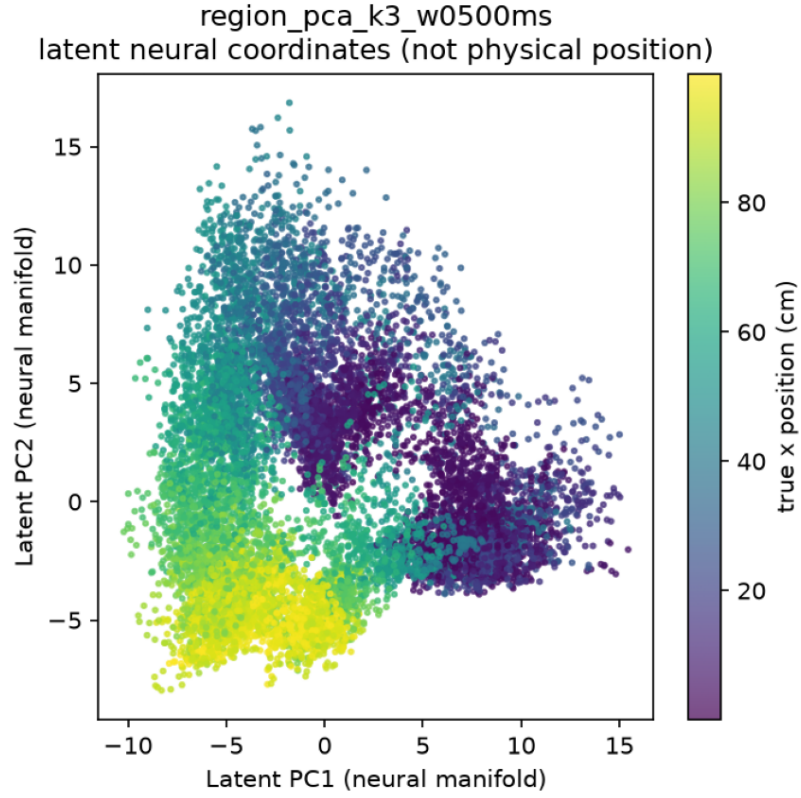


Figure 8: Population trajectory in the first two region-PCA coordinates, colored by true horizontal position. The structured color gradient indicates that the region-restricted latent space preserves spatial information.

13.3 Offline decoder selection and recording degradation

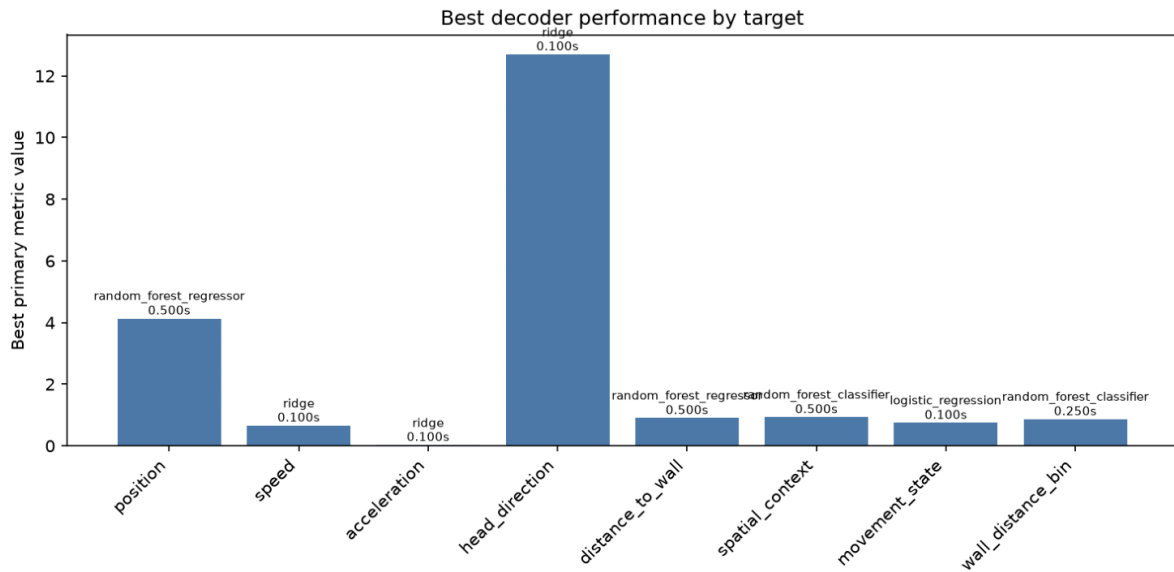


Figure 9: Best offline decoder and integration window selected for each behavioral target using ground-truth spikes.

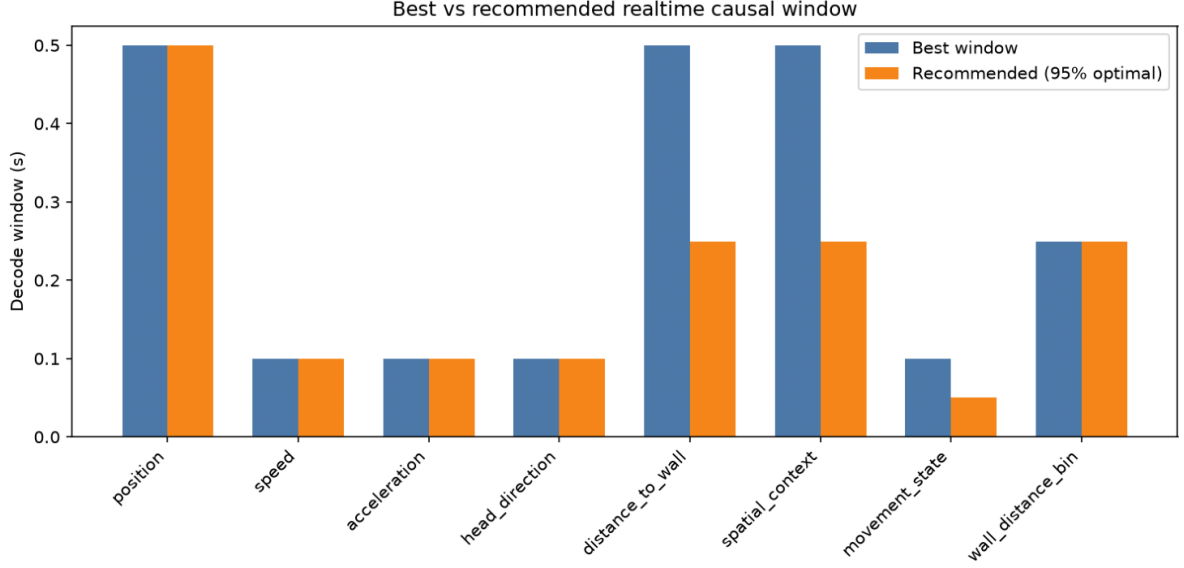
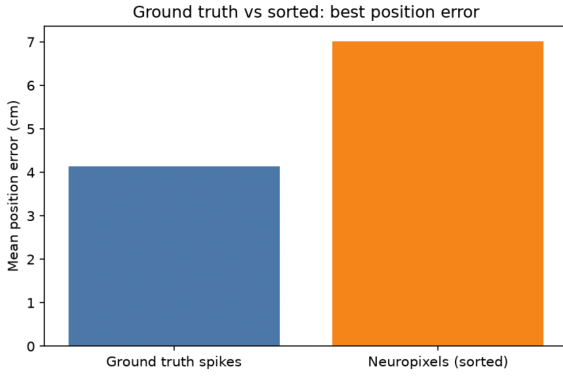
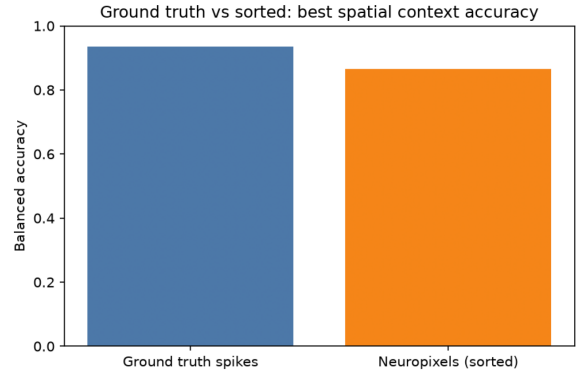


Figure 10: Best and shortest-near-optimal causal windows for each target. The selection policy can substantially reduce latency for distance-to-wall, spatial-context, and movement-state decoding.



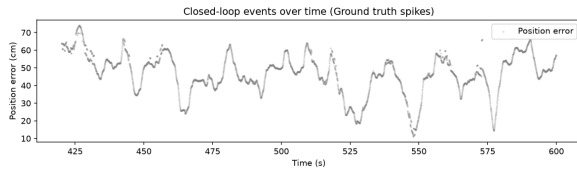
(a) Mean position error.



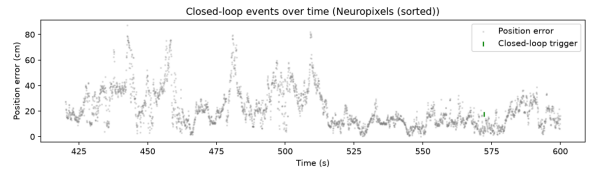
(b) Spatial-context balanced accuracy.

Figure 11: Effect of Neuropixels-like degradation and sorting on the best offline position and spatial-context decoders.

13.4 Causal realtime replay



(a) Ground-truth spike source.



(b) Sorted spike source.

Figure 12: Closed-loop events overlaid on causal realtime position error. Green and red markers indicate correct and incorrect triggers under the configured policy.

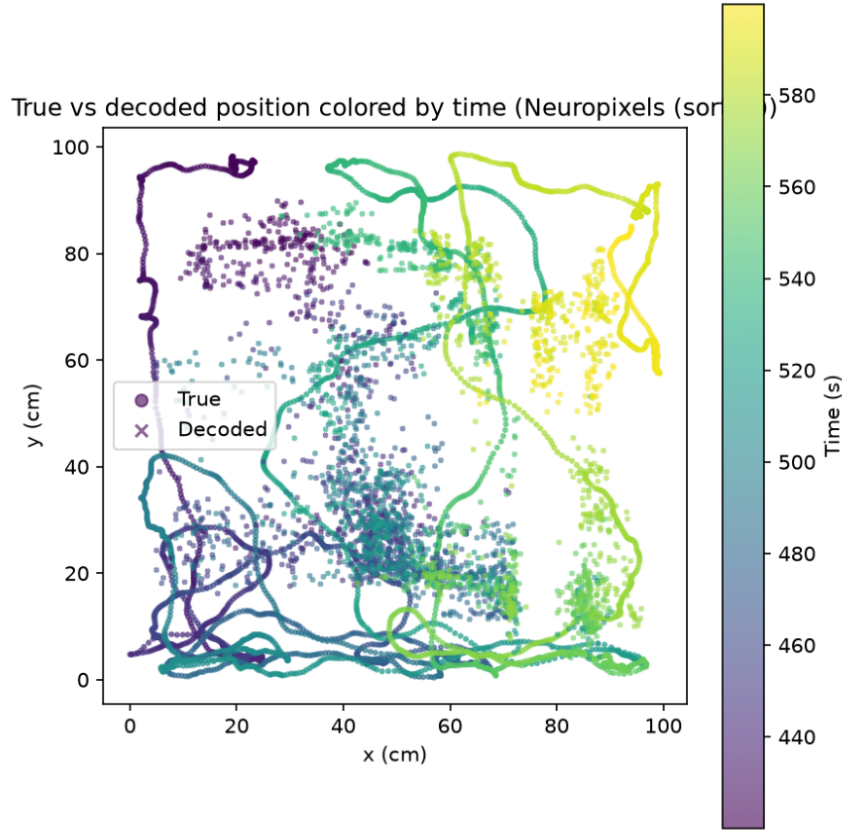


Figure 13: True and causally decoded position during realtime replay using Neuropixels-like sorted spikes, with both trajectories colored by time.

14 Outputs and visualization

Listing 2: Experiment directory layout.

```
outputs/<run>/
  behavior.csv, units.csv, spikes_*.csv, summary.json
  decoder_comparison/{sorted,ground_truth}/
  realtime_decoding/{spike_source}/{target}_{policy}/
  decoding/ # optional temporal W x L
  figures/
    decoder_comparison/
    realtime_decoding/
    output.pdf
```

`run.visualizations.py` detects available folders and plots simulation, comparison, and realtime figures without recomputing models. With `--compile-pdf`, all PNGs are assembled into a sectioned `figures/output.pdf`.

15 Future RatInABox behavior extensions

These notes record alignment with RatInABox and optional next steps for behavior generation. They are intended as a pickup brief for later implementation; none of the items below are required for the current decoder baseline.

15.1 Alignment with RatInABox

Hippo is a hippocampal data-generation pipeline (RatInABox locomotion $\rightarrow$ rate/spike models $\rightarrow$ sorting degradation $\rightarrow$ TCN/decoder training) and falls under RatInABox use case (i): generating realistic trajectories and/or neural data for downstream analysis and model training. Behavior is currently RatInABox random open-field motion; the `ratinbox_neurons` backend already syncs cell firing to a saved `BehaviorTrace` via `_sync_agent_to_behavior_step`, so once a trajectory exists—random, imported, or RL-generated—place/grid/HD cells can follow it. Only a behavior front-end (and optional RL loop) is missing. The success criterion is decoder (and later TCN) performance at deployment. Priority for that goal: match deployment data (especially real sorted spikes + that session’s tracking) $\gg$ sorting/noise realism $\gg$ imported or RL trajectories. Trajectory import and reward RL do *not* fix remapping, cross-day unit drift, or sorting contamination.

15.2 Real trajectory implementation (`import_trajectory`)

RatInABox can drive cells from an imported path without the random motion model (`Agent.import_trajectory(positions)`), or a `.npz` with keys `t/pos`, e.g. Sargolini for smoke tests). Users still call `Agent.update(dt)`, which interpolates along the path and *loops* if the run exceeds the imported duration. In Hippo this means a new behavior path (load real (t, xy)) while leaving rates $\rightarrow$ spikes $\rightarrow$ sorting $\rightarrow$ `ManifoldDataset` / decoders unchanged. Implementation pitfalls: keep arena scale consistent (meters in RatInABox, cm elsewhere); prefer `interpolate=False` for high-rate tracking; require solid boundaries; avoid short looping paths that bias training; imported/forced motion can illegally cross walls or leave the environment. Distinguish two meanings of “session trajectory”: (A) import a path into the sim to drive *synthetic* spikes—needs `import_trajectory`; (B) train on *real* sorted spikes aligned to that session’s tracking—standard alignment, no RatInABox import. Worth doing eventually for sim $\rightarrow$ real kinematic match (occupancy, speed/dwell, wall bias; especially for TCNs and speed/movement targets), but not the highest-leverage next step for decoder success.

15.3 RL-based reward simulation

Separately, RatInABox’s model-free RL demo (paper Fig. 3e) uses continuous TD learning: a value neuron approximates

$$V^\pi(x) = \sum_i w_i F_i^{\text{pc}}(x)$$

over place-cell features, then an improved policy sets `drift_velocity` $\propto \nabla V$ so the Agent biases toward high-value regions; iterating yields near-shortest paths to reward while avoiding walls. Hippo could use contris such as `ValueNeuron` and `TaskEnvironment` (plus `Agent.update(..., drift_velocity=...)`) to synthesize goal-directed sessions instead of random forage—changing occupancy near reward, approach/leave speeds, and stereotyped routes. Light reward stubs already exist (`reward_zones` viz, `reward_tuning_strength` metadata) but are not an RL system. This improves behavioral realism for reward-guided tasks and future reward-related labels, but only indirectly helps decoders; it does not train the spike-sorted TCN/decoder stack. Treat as a behavior-model feature when deployment or science needs learned reward-seeking—not a general decoder boost. Relative to import: `import_trajectory` replays a fixed recorded policy; TD+drift synthesizes a learned policy toward reward.

16 Limitations

- Simulated region functions and rate parameters are configurable hypotheses, not recovered biology.

- MEC/subiculum probe bands are co-recording conveniences for afferent cell types, not a claim about single-shank span in vivo.
- Inter-region feedforward (Eqs. 19–21) is derived but not yet implemented; only shared state, CA3 recurrence, and CA1.int anti-CA1 coupling are live.
- Sorted spikes lose some ground-truth metadata (identity, projections).
- Supervised embeddings (when added) reflect supplied labels and should not be treated as complete intrinsic geometry.
- Realtime-compatible methods must not use future spikes or future smoothing.
- Simulation results require later experimental validation.

17 Repository map (selected)

Table 9: Selected modules.

Path	Role
<code>hippo_sim/hippocampal_populations.py</code>	Maximally hippocampal RiaB population table
<code>hippo_sim/ratinabox_neural_backend.py</code>	RiaB rates + dynamics overlays
<code>hippo_sim/anatomy.py</code>	Probe segments and cell-type contact mapping
<code>hippo_sim/rate_equations.py</code>	Custom rate ODE backend
<code>hippo_sim/config.py</code>	Geometry, rate, recording, RiaB defaults
<code>realtime/timing.py</code>	dt_{update} , W , L , τ
<code>realtime/spike_features.py</code>	Causal count matrices
<code>realtime/decoder_models.py</code>	Model zoo
<code>realtime/bayesian_decoder.py</code>	Bayesian place decoder
<code>realtime/manifold_features.py</code>	Static manifold modes
<code>realtime/manifolds/</code>	Temporal raw / PCA encoders
<code>realtime/temporal/</code>	History models and $W \times L$ search
<code>realtime/workflow.py</code>	Public decoder orchestration
<code>visualization/</code>	Figures and PDF compilation

License: research / educational use.

Companion README: project root `README.md` (Unicode equations for Markdown preview).